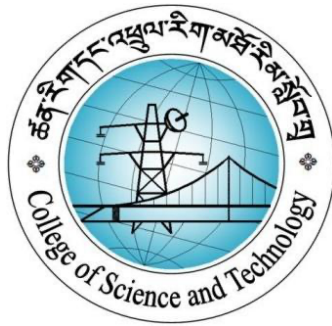


SWE 101 Report



Second Year Semester I

By:

Mr./Ms. Lhundup Dorji

Student Number: 02240349

Bachelor of Engineering in SOFTWARE ENGINEERING

COMPUTING TECHNOLOGIES DEPARTMENT

COLLEGE OF SCIENCE AND TECHNOLOGY

ROYAL UNIVERSITY OF BHUTAN

PHUENTSHOLING, BHUTAN

3rd MAY 2026.

1. Introduction

This report documents the implementation of PushDemo, a mobile application built using React Native and the Expo framework as part of the SWE 101 practical. The project focuses on push notifications — a feature that allows a server to alert a user's device even when the app is not open. The application was developed following a step-by-step guide and covers the full notification lifecycle, from requesting device permission to receiving a remote push sent through the Expo browser tool.

2. Objective of the Project

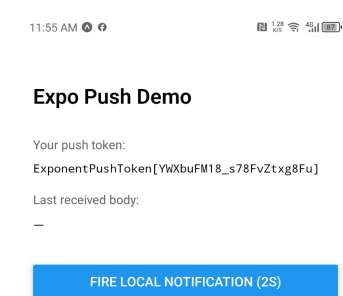
The objectives of this practical were to understand how Expo's push notification system works, to register a physical device and obtain an Expo push token, to configure platform-specific settings for both Android and iOS, to implement foreground and background notification handlers, and to test the complete flow by sending a notification through the Expo Push Notification Tool in the browser.

3. Tools and Setup

Before writing any code, the following tools and accounts were arranged. Node.js 20 LTS was installed from nodejs.org, which also provided npm. The EAS CLI was then installed globally using `npm install -g eas-cli`. A free Expo account was created at expo.dev/signup, which was needed to obtain the project ID and to use the Expo Push Notification Tool for testing. A physical Android device with Expo Go installed was used for running the application.

4. Project Setup

The project was created using `npx create-expo-app@latest PushDemo` with the Blank TypeScript template. Three packages were then installed using `npx expo install expo-notifications expo-device expo-constants`. After logging in with `eas login`, the command `eas init` was run to register the project with Expo's servers. This automatically inserted a UUID into `app.json` under `extra.eas.projectId`. The `app.json` file was then updated to include the Android package name, the iOS bundle identifier, and the `expo-notifications` plugin entry.



5. Implementation

The entire application logic lives in `App.tsx` and is organised into three parts.

The first part is a global notification handler set outside the component using `Notifications.setNotificationHandler()`. Being at the module level ensures it is in place before any notification arrives. It configures foreground behaviour to show alerts, play sound, and display banners.

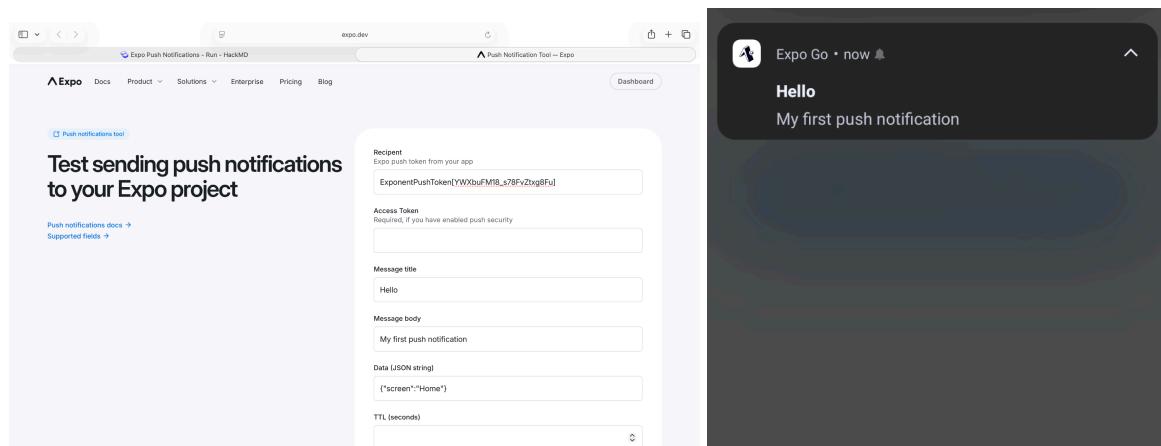
The second part is a registration function, `registerForPushNotificationsAsync()`, which runs when the app mounts. It checks that the app is on a real device, creates an Android

notification channel, requests permission from the operating system, reads the project ID from the Expo configuration, and then calls `Notifications.getExpoPushTokenAsync()` to obtain the device token from Expo's servers.

The third part is the `App` component itself. It stores the push token and the body of the last received notification in state. Inside a `useEffect`, it calls the registration function and attaches two event listeners using refs. The received listener updates the displayed message whenever a notification arrives while the app is open. The response listener fires when the user taps a notification from any app state, including background and cold-start, and shows an alert with the notification's data payload. Both listeners are removed cleanly on unmount. A button on the screen also triggers a local notification scheduled two seconds in the future.

6. Testing

Testing was done on a physical Android device running the app through Expo Go. Once the app launched and permission was granted, the Expo push token appeared on screen. This token was copied and pasted into the Expo Push Notification Tool at expo.dev/notifications. The title was set to "Hello", the body to "My first push notification", and the data payload to `{"screen": "Home"}`. After clicking Send, the phone was locked so the app moved to the background. Within a few seconds the notification appeared on the lock screen. Tapping it brought the app forward and displayed the data payload in an alert, confirming the response listener was working correctly. The local notification button was also tested and the notification appeared after the two-second delay as expected.



7. Challenges Faced

The first challenge was discovering that push notifications do not work on a simulator, so a physical device was required throughout. The second challenge was the ordering requirement on Android: the notification channel must be created before requesting the token, and getting this wrong produced an unclear error. The third challenge involved reading the project ID at runtime, as the path to this value differs across Expo SDK versions. The code handles this using a nullish coalescing chain to check both the current and legacy config paths.

8. Conclusion

PushDemo successfully demonstrates the push notification lifecycle using Expo. The practical covered permission handling, device registration, platform-specific configuration, event listener management, local notification scheduling, and remote delivery through the Expo browser tool. Working through the implementation on a real device gave a much clearer understanding of how push notifications function than reading documentation alone would have provided. The patterns used here, particularly around listener cleanup and platform branching, apply directly to any production React Native application.